

Bihar Yaatra

Full-Stack System Architecture

Next.js · Express.js · Supabase · JWT · Razorpay

Document Type: System Architecture & Sprint Plan (v3.0)
Project: Bihar Yaatra — Bihar Tourism Platform (Full Rebuild)
Frontend: Next.js 14 (App Router) + TailwindCSS + shadcn/ui
Backend: Express.js REST API + JWT Authentication
Database: Supabase (PostgreSQL) + Row Level Security
Storage: Supabase Storage (images & media)
Payments: Razorpay (INR gateway)
AI: Google Gemini AI (Saarathi Trip Planner)
Deployment: Vercel (Frontend) + Railway (API) + Supabase Cloud
Total Est. Build: 21 – 28 Days (AI-accelerated, 1–2 developers)

Confidential — Internal Use Only

1. Executive Summary

This document defines the complete system architecture for Bihar Yaatra v3.0 — a ground-up rebuild of the platform using a modern production-grade full-stack: Next.js 14 (App Router) for the frontend, Express.js for a dedicated REST API backend, Supabase (PostgreSQL) as the database, JWT for stateless authentication, and Razorpay for INR payments.

The rebuild replaces the Firebase-only architecture with a more scalable, maintainable, and interview-ready stack that gives the team full control over the API layer, database schema, and business logic. Supabase provides the managed PostgreSQL database with built-in Row Level Security, Auth helpers, and S3-compatible object storage, eliminating the need to manage raw infrastructure.

WHY THIS STACK?

Next.js 14 (App Router) → Server Components, built-in SEO, ISR, API routes — the industry default for production React apps

Express.js → Full control over REST API design, middleware, business logic, and Razorpay webhook handling

Supabase → Managed PostgreSQL with Realtime, Row Level Security, Auth (email/OAuth/OTP), and S3 Storage

JWT → Stateless token auth: Access Token (15 min) + Refresh Token (7 days), stored in httpOnly cookies

Razorpay → INR-native payment gateway, webhook-driven order lifecycle, zero monthly fees

Gemini AI Action → Saarthi trip planner — already working, migrates as a Next.js Server Action

1.1 Build Timeline at a Glance

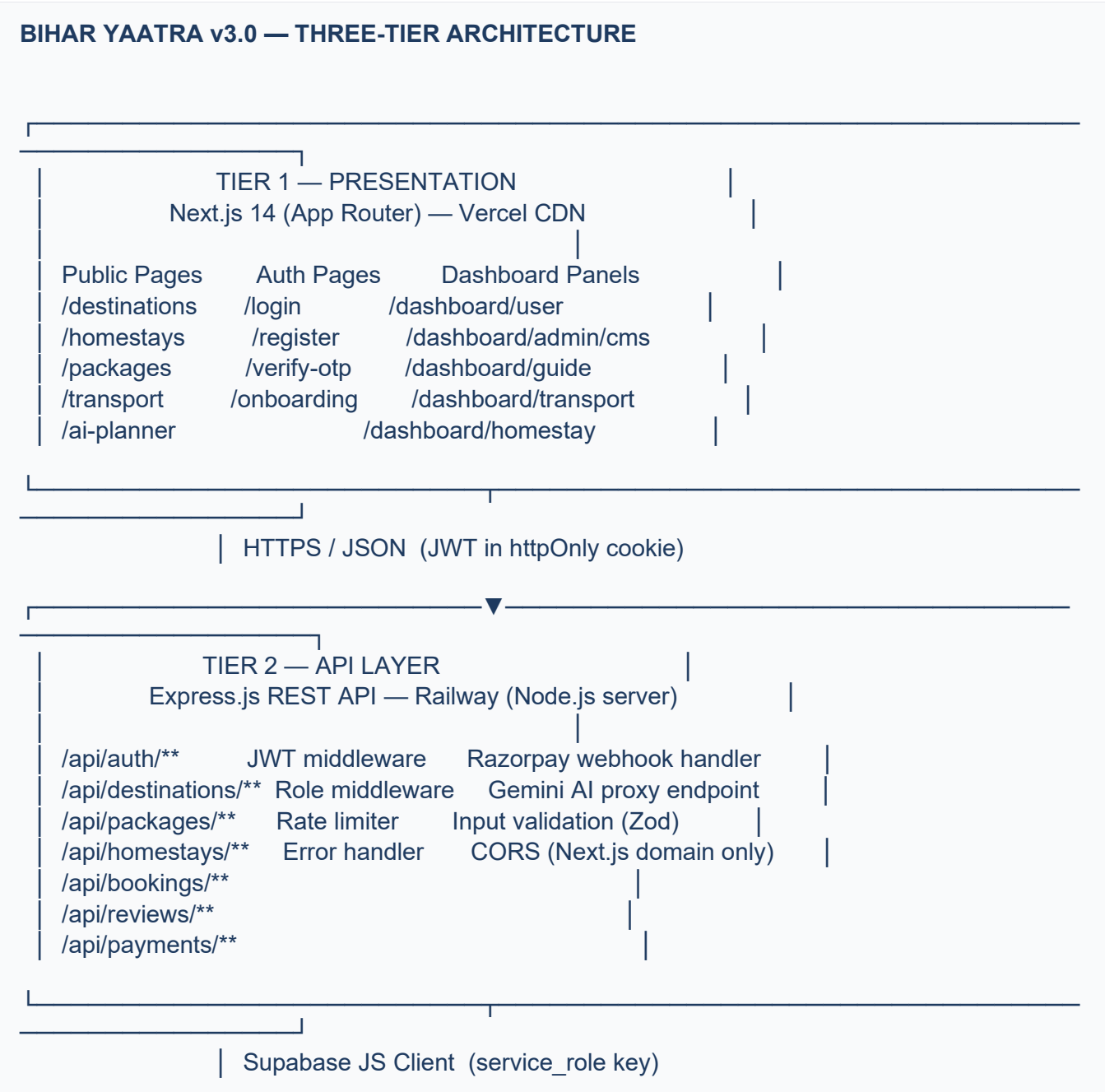
Week	Phase	Key Deliverables	Est. Hours	Status
W1	Project Setup + Auth	Supabase schema, Express API skeleton, JWT auth (register/login/refresh/logout), Next.js App Router scaffold, all auth pages	30 – 36 hrs	PLANNED
W2	Core Content + CMS	Destinations, Packages, Homestays, Transport, Guides — CRUD API endpoints + Next.js pages + Admin CMS panel	36 – 42 hrs	PLANNED
W3	Booking + Payments + Reviews	Razorpay order flow, webhook handler, booking management, review & rating system, user dashboard	34 – 40 hrs	PLANNED
W4	Polish + Security + Deploy	Row Level Security, Supabase Storage, Gemini AI (Saarthi), SEO, Google Maps, Vercel + Railway deploy, CI/CD	28 – 34 hrs	PLANNED

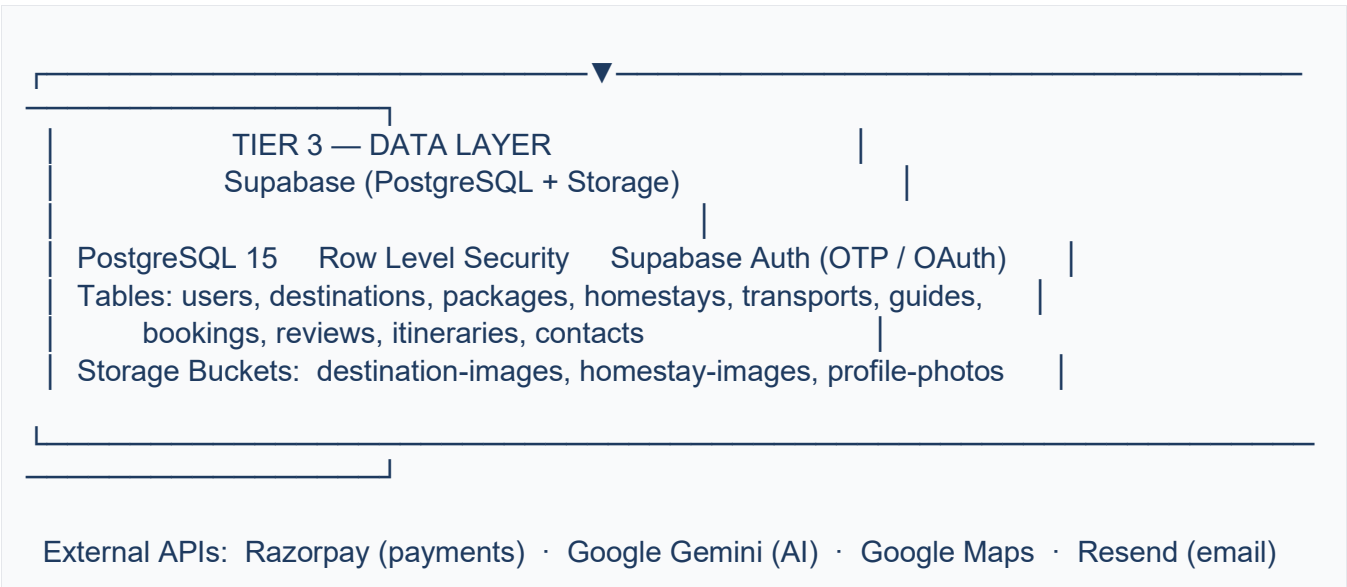
Total estimated effort: 128 – 152 hours across 4 weeks | 1 developer: 4 weeks | 2 developers: 2.5 weeks

2. System Architecture

Bihar Yaatra v3.0 uses a decoupled three-tier architecture. The Next.js frontend communicates exclusively with the Express.js API via HTTP/JSON. The Express API is the sole client of the Supabase PostgreSQL database and Supabase Storage. This separation means the frontend is never directly coupled to the database — all data access is mediated through authenticated API routes.

2.1 Architecture Diagram





2.2 Request Lifecycle

EXAMPLE: User books a homestay

1. Next.js page calls POST /api/bookings (httpOnly cookie carries JWT)
2. Express JWT middleware verifies access token → attaches req.user
3. Role middleware confirms user.role === 'traveller'
4. Zod schema validates request body (homestay_id, check_in, check_out, guests)
5. Service layer calls Supabase to check homestay availability
6. Razorpay order created → order_id returned to client
7. Client completes Razorpay checkout in browser
8. Razorpay fires webhook → POST /api/payments/webhook
9. Express verifies Razorpay signature → inserts booking row in Supabase
10. Resend sends confirmation email via API
11. Next.js revalidates /dashboard/user/bookings via revalidatePath()

3. Complete Technology Stack

Layer	Technology	Tool / Package	Purpose
Frontend	Next.js 14	App Router, Server Components	Pages, layouts, SSR/ISR, SEO meta, API routes
Frontend	React 18	Hooks, Context, Suspense	Component model and client interactivity
Frontend	TailwindCSS v3	Utility classes + custom theme	Responsive styling (saffron brand colours)
Frontend	shadcn/ui	Radix UI primitives	Accessible component library (modals, dropdowns)
Frontend	Zustand	Client state management	Auth state, cart, UI toggles
Frontend	React Hook Form + Zod	Form state + schema validation	All forms — client-side validation
Frontend	TanStack Query	Server state, caching	API data fetching, mutation, cache invalidation
Frontend	Next/Image	Built-in image component	Optimized WebP delivery from Supabase Storage
Frontend	Lucide React	Icon library	Consistent SVG icons platform-wide
Backend	Express.js 4	Node.js web framework	REST API server — all business logic
Backend	TypeScript	Static typing	Type-safe API code, shared types with frontend
Backend	Zod	Schema validation	Request body validation on all POST/PUT endpoints
Backend	jsonwebtoken	JWT sign/verify	Access token (15m) + Refresh token (7d) flow
Backend	bcryptjs	Password hashing	Secure password storage in users table
Backend	cors	CORS middleware	Restrict API to Next.js domain only
Backend	helmet	Security headers	XSS, CSRF, MIME-type protection
Backend	express-rate-limit	Rate limiting	100 req/min general, 10 req/min on auth routes
Backend	morgan	HTTP request logger	Development + production access logs
Database	Supabase	Managed PostgreSQL 15	All platform data — fully relational schema
Database	@supabase/supabase-js	Supabase JS client	DB queries, Realtime subscriptions
Database	Row Level Security	Supabase RLS policies	Data-level access control per user/role
Storage	Supabase Storage	S3-compatible object store	Images for destinations, homestays, profiles

Layer	Technology	Tool / Package	Purpose
Auth	Supabase Auth	Email · Google OAuth · OTP	User identity, session management
Auth	JWT (custom)	Express middleware	Stateless API auth — decoupled from Supabase session
Payments	Razorpay	Payment gateway	INR bookings — order create + webhook + refund
AI	Google Gemini AI	@google/generative-ai	Saarthi trip planner — personalised itineraries
Maps	Google Maps API	Maps JS API	Interactive destination + homestay maps
Email	Resend	resend npm package	Booking confirmations, OTP emails
Deployment	Vercel	Vercel CLI / GitHub Action	Next.js frontend — global edge CDN
Deployment	Railway	Dockerfile + GitHub Action	Express.js API — auto-scale Node.js
DevOps	GitHub Actions	CI/CD workflow	Lint + test + deploy on push to main
DevOps	ESLint + Prettier	Code quality	Consistent formatting across mono-repo

4. Authentication & JWT Flow

Authentication uses a dual-token JWT strategy. On successful login, the Express API issues an Access Token (short-lived, 15 minutes) and a Refresh Token (long-lived, 7 days). Both are stored in httpOnly, Secure, SameSite=Strict cookies — never in localStorage, making them immune to XSS attacks.

4.1 Token Strategy

Property	Access Token	Refresh Token
Library	jsonwebtoken (Express signs both)	
Expiry	15 minutes	7 days
Storage	httpOnly cookie (JS cannot read)	httpOnly cookie
Payload	{ user_id, email, role, iat, exp }	{ user_id, token_family }
Secret	JWT_ACCESS_SECRET (env var) JWT_REFRESH_SECRET (env var)	
Rotation	Not rotated	Rotated on every refresh
Revocation	Stateless — expiry only invalidated	Stored in DB — can be

4.2 Auth Flow Diagram

REGISTRATION FLOW

POST /api/auth/register { name, email, password, phone }

→ bcrypt.hash(password) → INSERT users row → send OTP via Supabase Auth

POST /api/auth/verify-otp { email, otp }

→ verify OTP → UPDATE users SET email_verified=true

→ sign access_token + refresh_token → set httpOnly cookies → 201

LOGIN FLOW

POST /api/auth/login { email, password }

→ SELECT user WHERE email → bcrypt.compare() → sign tokens → set cookies → 200

TOKEN REFRESH FLOW (called by Next.js middleware on 401)

POST /api/auth/refresh (refresh_token cookie sent automatically)

→ verify refresh_token → rotate token → set new cookies → 200

LOGOUT FLOW

POST /api/auth/logout → DELETE refresh_token from DB → clear cookies → 200

GOOGLE OAUTH FLOW

GET /api/auth/google → redirect to Google → callback sets JWT cookies

4.3 User Roles

Role	Description
guest	Unauthenticated visitor. Can browse public pages. No API write access.
traveller	Registered user. Can book, write reviews, save AI itineraries, manage profile.
provider	Homestay owner / Guide / Transport operator. Manages own listings & bookings.
admin	Full CMS access. Approve providers, view all bookings, manage platform content.
superadmin	All admin rights + manage other admins, billing dashboard, system config.

5. Supabase (PostgreSQL) Database Schema

All tables live in the public schema of the Supabase PostgreSQL instance. Foreign keys enforce referential integrity. Row Level Security (RLS) policies — defined in Section 5.11 — provide fine-grained access control directly at the database layer.

5.1 Table Overview

Table	Description
users	Platform user accounts — all roles stored here with a role column
refresh_tokens	Active refresh tokens for JWT revocation support
destinations	Bihar tourist destinations with rich article content (JSON sections)
packages	Tour packages with day-wise itinerary (JSON array)
homestays	Homestay listings with amenities (JSON array) and host reference
transports	Transport listings — vehicles available for hire
guides	Local guide profiles with skills, languages, and verification status
bookings	All booking records — unified across package/homestay/transport/guide
reviews	Verified traveller reviews for any service type
saved_itineraries	AI-generated trip plans saved by travellers (Saarthi)
contacts	Contact form submissions
provider_applications	Partner / guide onboarding applications pending admin review

Table: **users**

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	Auto-generated primary key (gen_random_uuid())
name	varchar(100)		NO	Full display name
email	varchar(255)		NO	Unique email address — login credential
password_hash	text		YES	bcrypt hash — NULL for OAuth-only users
phone	varchar(15)		YES	Mobile number for OTP login
role	varchar(20)		NO	Enum: guest traveller provider admin superadmin
email_verified	boolean		NO	TRUE after OTP or OAuth verification

Column	Type	PK / FK	Nullable	Description
avatar_url	text		YES	Supabase Storage URL
provider_type	varchar(20)		YES	NULL homestay guide transport
is_active	boolean		NO	Admin can deactivate accounts
created_at	timestampz		NO	DEFAULT now()
updated_at	timestampz		NO	DEFAULT now() — trigger auto-updates

Table: **destinations**

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
name	varchar(150)		NO	Full destination name
slug	varchar(150)		NO	UNIQUE — URL-safe identifier
tagline	varchar(255)		YES	Short marketing headline
category	varchar(50)		NO	heritage spiritual nature cultural
location	varchar(100)		NO	District / region in Bihar
hero_image_url	text		YES	Supabase Storage URL
sections	jsonb		YES	[{header, content}] Wikipedia-style article
highlights	jsonb		YES	[string] — top highlights array
best_time	varchar(100)		YES	E.g. October to February
lat	decimal(9,6)		YES	Latitude for Google Maps pin
lng	decimal(9,6)		YES	Longitude for Google Maps pin
tags	jsonb		YES	[string] — search / filter tags
is_published	boolean		NO	FALSE = draft, TRUE = live on site
created_by	uuid	FK→users.id	NO	Admin UID who created this record
created_at	timestampz		NO	DEFAULT now()

Table: **packages**

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
title	varchar(200)		NO	Package display name
slug	varchar(200)		NO	UNIQUE URL slug
duration_days	integer		NO	Total trip days
duration_nights	integer		NO	Total nights
price_per_person	numeric(10,2)		NO	Base price in INR
cover_image_url	text		YES	Supabase Storage URL
destination_ids	jsonb		NO	[uuid] — array of destination IDs

Column	Type	PK / FK	Nullable	Description
itinerary	jsonb		NO	[[day, title, description, meals]]
includes	jsonb		YES	[string] — inclusions list
excludes	jsonb		YES	[string] — exclusions list
max_group_size	integer		YES	Max travellers per booking
difficulty	varchar(20)		YES	easy moderate challenging
is_published	boolean		NO	DEFAULT false
created_by	uuid	FK→users.id	NO	Admin who created record
created_at	timestampz		NO	DEFAULT now()

Table: **homestays**

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
name	varchar(150)		NO	Property name
slug	varchar(150)		NO	UNIQUE URL slug
location	varchar(100)		NO	Village / district
address	text		YES	Full address for maps
lat	decimal(9,6)		YES	Google Maps latitude
lng	decimal(9,6)		YES	Google Maps longitude
price_per_night	numeric(10,2)		NO	INR per night
max_guests	integer		NO	Maximum occupancy
amenities	jsonb		NO	[string] wifi kitchen ac parking meals
images	jsonb		YES	[string] array of Supabase Storage URLs
host_id	uuid	FK→users.id	NO	Provider user UID
is_available	boolean		NO	Real-time availability toggle
is_published	boolean		NO	Admin approval status
avg_rating	numeric(3,2)		YES	Cached aggregate rating (1.00–5.00)
review_count	integer		NO	DEFAULT 0 — updated by trigger
created_at	timestampz		NO	DEFAULT now()

Table: **bookings**

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
user_id	uuid	FK→users.id	NO	Traveller who made booking
service_type	varchar(20)		NO	package homestay transport guide
service_id	uuid		NO	ID in the relevant service table

Column	Type	PK / FK	Nullable	Description
service_name	varchar(200)		NO	Denormalized name for display
check_in	date		YES	Start date of stay / trip
check_out	date		YES	End date of stay / trip
guests	integer		NO	Number of travellers
total_amount	numeric(10,2)		NO	Final price charged in INR
status	varchar(20)		NO	pending confirmed cancelled completed
razorpay_order_id	varchar(100)		YES	Razorpay order reference
razorpay_payment_id	varchar(100)		YES	Razorpay payment reference (post-success)
payment_status	varchar(20)		NO	unpaid paid refunded
notes	text		YES	Special requests from traveller
created_at	timestampz		NO	DEFAULT now()

Table: reviews

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
service_type	varchar(20)		NO	package homestay transport guide
service_id	uuid		NO	ID of reviewed service
user_id	uuid	FK→users.id	NO	Reviewer UID
booking_id	uuid	FK→bookings.id	YES	Linked completed booking — enforces verified reviews
rating	integer		NO	CHECK (rating >= 1 AND rating <= 5)
comment	text		YES	Written review text
is_verified	boolean		NO	TRUE if booking_id is not null and booking is completed
created_at	timestampz		NO	DEFAULT now()

Table: saved_itineraries

Column	Type	PK / FK	Nullable	Description
id	uuid	PK	NO	gen_random_uuid()
user_id	uuid	FK→users.id	NO	Owner traveller UID
title	varchar(200)		NO	User-defined trip name
preferences	jsonb		NO	{budget, days, interests, group_type, from_location}
itinerary	jsonb		NO	Full Gemini AI response — day-wise plan
created_at	timestampz		NO	DEFAULT now()

5.11 Supabase Row Level Security Policies

RLS POLICY SUMMARY — KEY RULES

destinations, packages, homestays, transports, guides:

SELECT → allow all (public read)

INSERT / UPDATE / DELETE → allow if (SELECT role FROM users WHERE id = auth.uid()) IN ('admin','superadmin')

bookings:

SELECT → allow if user_id = auth.uid() OR role IN ('admin','provider')

INSERT → allow if auth.uid() IS NOT NULL

UPDATE → allow if role IN ('admin') OR (user_id = auth.uid() AND status = 'pending')

reviews:

SELECT → allow all

INSERT → allow if auth.uid() IS NOT NULL

AND NOT EXISTS (SELECT 1 FROM reviews WHERE user_id=auth.uid() AND service_id=NEW.service_id)

UPDATE / DELETE → allow if user_id = auth.uid() OR role = 'admin'

saved_itineraries:

ALL operations → allow if user_id = auth.uid()

users:

SELECT → allow own row OR role = 'admin'

UPDATE → allow own row (limited columns) OR role = 'admin'

6. Express.js REST API Design

The Express API is the single source of truth for all data mutations and business logic. It uses a layered architecture: Router → Middleware → Controller → Service → Supabase Client. All routes are versioned under `/api/v1/`. TypeScript interfaces are shared with the Next.js frontend via a shared `/types` package.

6.1 Project Structure — Express API

/api (Express.js project)

```

src/
├── server.ts           ← Express app init, global middleware
├── config/
│   ├── supabase.ts    ← Supabase client (service_role key)
│   ├── jwt.ts         ← Token sign/verify helpers
│   └── razorpay.ts     ← Razorpay instance config
├── middleware/
│   ├── authenticate.ts ← Verify JWT → attach req.user
│   ├── authorize.ts    ← Check req.user.role against allowed roles
│   ├── validate.ts     ← Zod schema middleware factory
│   └── rateLimiter.ts  ← express-rate-limit config
├── routes/
│   ├── auth.routes.ts  ← /api/v1/auth/**
│   ├── destinations.routes.ts
│   ├── packages.routes.ts
│   ├── homestays.routes.ts
│   ├── transports.routes.ts
│   ├── guides.routes.ts
│   ├── bookings.routes.ts
│   ├── reviews.routes.ts
│   ├── payments.routes.ts ← Razorpay order + webhook
│   ├── ai.routes.ts    ← Gemini AI proxy
│   └── upload.routes.ts ← Supabase Storage signed URL
├── controllers/        ← One controller per resource
├── services/           ← Business logic + Supabase queries
├── schemas/            ← Zod validation schemas
└── types/              ← Shared TypeScript interfaces
  
```

6.2 API Endpoint Reference

Auth Routes — `/api/v1/auth`

Method	Route	Purpose	Auth	Response
POST	/register	Create new user account	Public	201 { message }
POST	/verify-otp	Verify email OTP	Public	200 + JWT cookies
POST	/login	Login with email + password	Public	200 + JWT cookies
POST	/refresh	Rotate JWT token pair	Public	200 + new cookies
POST	/logout	Invalidate refresh token	JWT	200 { message }
GET	/me	Get authenticated user profile	JWT	200 { user }
POST	/forgot-password	Send password reset email	Public	200 { message }
POST	/reset-password	Set new password with token	Public	200 { message }
GET	/google	Google OAuth redirect	Public	302 redirect
GET	/google/callback	Google OAuth callback	Public	302 + JWT cookies

Destinations — /api/v1/destinations

Method	Route	Purpose	Auth	Response
GET	/	List all published destinations	Public	200 [destination]
GET	/:slug	Get destination by slug	Public	200 { destination }
POST	/	Create destination	Admin	201 { destination }
PUT	/:id	Update destination	Admin	200 { destination }
DELETE	/:id	Delete destination	Admin	204
GET	/search?q=	Search destinations by name/tag	Public	200 [destination]

Homestays — /api/v1/homestays

Method	Route	Purpose	Auth	Response
GET	/	List all available homestays	Public	200 [homestay]
GET	/:slug	Get homestay detail	Public	200 { homestay, reviews }
POST	/	Create homestay listing	JWT	201 { homestay }
PUT	/:id	Update own homestay	JWT	200 { homestay }
DELETE	/:id	Delete homestay	JWT	204
PATCH	/:id/availability	Toggle availability	JWT	200 { is_available }
GET	/my	Get provider's own listings	JWT	200 [homestay]

Bookings — /api/v1/bookings

Method	Route	Purpose	Auth	Response
POST	/	Create booking (after payment)	JWT	201 { booking }
GET	/my	Get own booking history	JWT	200 [booking]
GET	/:id	Get single booking detail	JWT	200 { booking }
PATCH	/:id/cancel	Cancel a pending booking	JWT	200 { booking }
GET	/	List all bookings (admin)	Admin	200 [booking]
PATCH	/:id/status	Update booking status	Admin	200 { booking }

Payments — /api/v1/payments

Method	Route	Purpose	Auth	Response
POST	/create-order	Create Razorpay order	JWT	201 { order_id, amount, currency }
POST	/webhook	Razorpay payment webhook	Public	200 { received }
POST	/verify	Client-side payment verification	JWT	200 { booking_id }
POST	/refund/:booking_id	Initiate refund	Admin	200 { refund_id }

Reviews — /api/v1/reviews

Method	Route	Purpose	Auth	Response
GET	/:service_type/:service_id	Get reviews for a service	Public	200 [review]
POST	/	Submit a review (verified booking required)	JWT	201 { review }
DELETE	/:id	Delete own review	JWT	204
DELETE	/admin/:id	Admin delete any review	Admin	204

AI — /api/v1/ai

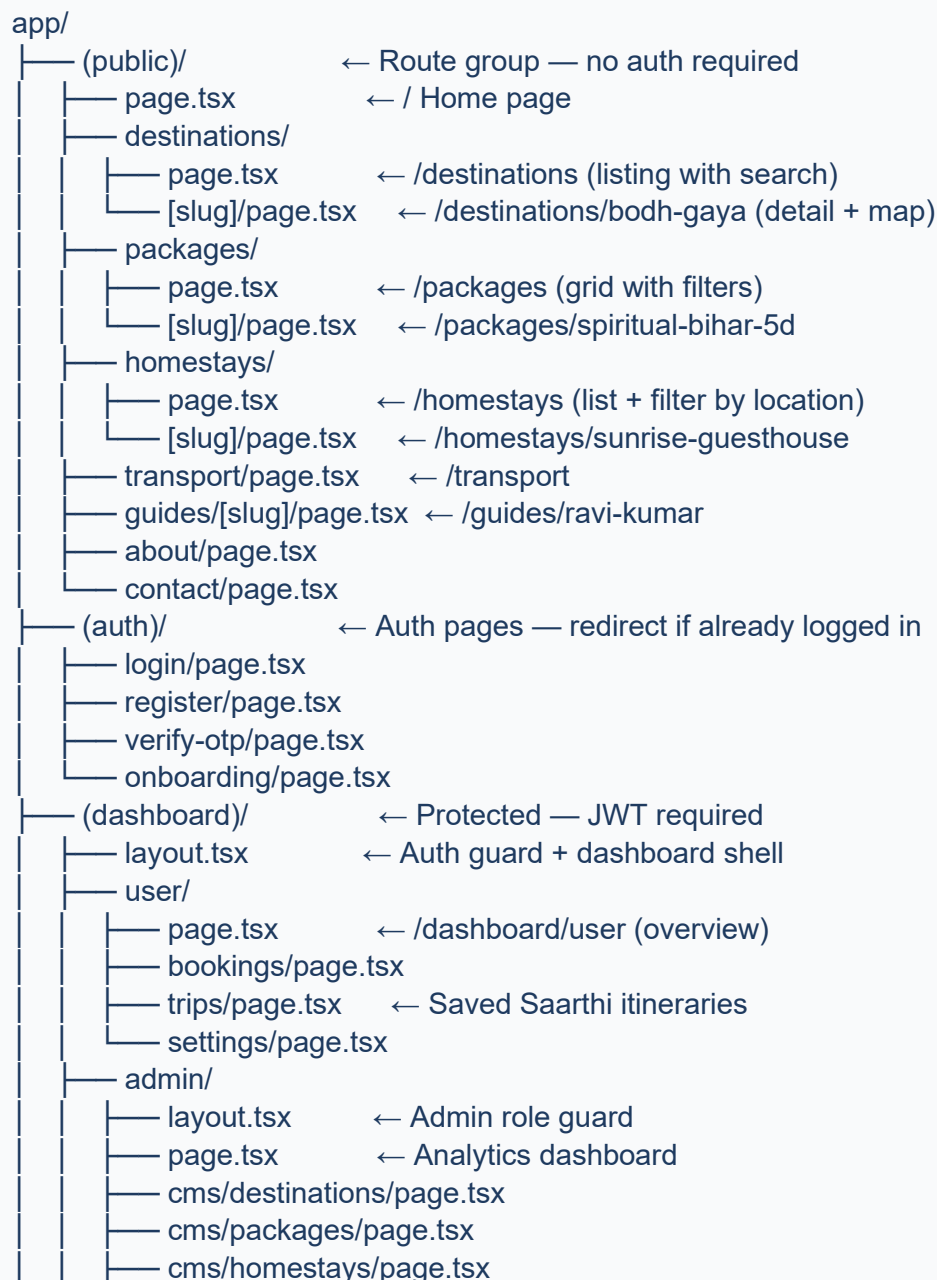
Method	Route	Purpose	Auth	Response
POST	/generate-itinerary	Generate Saarthi trip plan	JWT	200 { itinerary }
POST	/save-itinerary	Save itinerary to user account	JWT	201 { id }
GET	/saved	Get all saved itineraries	JWT	200 [itinerary]
DELETE	/saved/:id	Delete saved itinerary	JWT	204

7. Next.js 14 Frontend Architecture

The frontend uses the Next.js 14 App Router. All pages that require authentication are Server Components that read the JWT from the request cookie and redirect to /login if invalid. Client Components (marked 'use client') handle interactive UI. TanStack Query manages all API state on the client.

7.1 App Router Directory Structure

/frontend (Next.js 14 project)



```

|
| |
| | | — cms/transport/page.tsx
| | | — cms/guides/page.tsx
| | | — bookings/page.tsx
| | — provider/
| | | — [type]/page.tsx ← /dashboard/provider/homestay|guide|transport
| | — ai-planner/page.tsx ← Saarthi trip planner
| — api/ ← Next.js API Routes
| | — auth/google/route.ts ← OAuth callback handler
| — layout.tsx ← Root layout — Providers, fonts
| — not-found.tsx

```

components/

```

| — ui/ ← shadcn/ui base components
| — layout/ ← Navbar, Footer, Sidebar, DashboardShell
| — destination/ ← DestinationCard, DestinationMap, ArticleSection
| — booking/ ← BookingForm, BookingCard, PaymentModal
| — review/ ← ReviewForm, ReviewList, StarRating
| — ai/ ← SaarthiForm, ItineraryCard, SavedTrips
| — common/ ← LoadingSpinner, EmptyState, ErrorBoundary

```

lib/

```

| — api-client.ts ← Axios instance with JWT cookie auto-attach
| — auth.ts ← Server-side JWT verify from cookie
| — utils.ts ← cn(), formatINR(), formatDate()

```

7.2 Server vs Client Component Strategy

Component Type	When to Use
Server Component (default)	Page layouts, data fetching on initial load, SEO-critical content. Reads JWT from cookie server-side to personalize content without a client round-trip.
Client Component ('use client')	Interactive UI: forms, modals, filters, dropdowns, the Saarthi AI planner (streaming), maps, real-time booking status.
Server Action	Form submissions where full page reload is acceptable: contact form, newsletter. Reduces client bundle size vs TanStack mutation.
API Route (/api/**)	OAuth callbacks only. All other data goes through Express API — not Next.js API routes.

8. Razorpay Payment Integration

Payments follow a server-confirmed webhook pattern. The client never directly updates the database — the booking record is only created after the Express webhook handler independently verifies the payment signature from Razorpay's server. This prevents any possibility of forged payment confirmations.

8.1 Payment Flow — Step by Step

RAZORPAY PAYMENT LIFECYCLE

STEP 1 — ORDER CREATION
Client → POST /api/v1/payments/create-order { service_id, service_type, guests, dates }
Express → validates booking params → calls razorpay.orders.create({ amount, currency:'INR' })
Express → saves pending booking in DB (status=pending, payment_status=unpaid)
Express → returns { order_id, amount, currency, booking_id } to client

STEP 2 — CLIENT CHECKOUT
Client → opens Razorpay checkout modal with order_id + amount
User → enters card / UPI / net banking details
Razorpay → charges user → fires webhook to Express

STEP 3 — WEBHOOK CONFIRMATION (server-to-server, cannot be forged)
Razorpay → POST /api/v1/payments/webhook { event: 'payment.captured', payload }
Express → verifies HMAC-SHA256 signature using RAZORPAY_WEBHOOK_SECRET
Express → updates booking: status=confirmed, payment_status=paid, razorpay_payment_id
Express → sends confirmation email via Resend
Express → returns 200 (Razorpay requires 200 within 5 seconds)

STEP 4 — CLIENT CONFIRMATION
Client → polls GET /api/v1/bookings/:id every 2s until status=confirmed
Client → redirects to /dashboard/user/bookings

8.2 Razorpay Environment Variables

Variable	Description
RAZORPAY_KEY_ID	Public key — safe to expose in client Razorpay checkout options
RAZORPAY_KEY_SECRET	Private key — only on Express server, never in frontend

RAZORPAY_WEBHOOK_SECRET	Webhook signing secret — used to verify POST /webhook HMAC
RAZORPAY_ACCOUNT_NUMBER	Settlement account for auto-transfers (live mode only)

9. AI Integration — Saarthi Trip Planner

Saarthi is the AI trip planner powered by Google Gemini. In v3.0, the Gemini API call moves from the browser (vulnerable to key exposure) to the Express backend. The `/api/v1/ai/generate-itinerary` endpoint proxies the call to Gemini, keeping the API key server-side. The Next.js client streams the response for a live typing effect.

9.1 Saarthi Architecture

SAARTHI FLOW — v3.0 (Secure Server-Side)

1. User fills SaarthiForm: budget, days, interests, from_location, group_type
2. Client → POST `/api/v1/ai/generate-itinerary` (JWT cookie auto-attached)
3. Express verifies JWT → builds structured Gemini prompt with Bihar context
4. Express calls Gemini API (GEMINI_API_KEY server env var — never exposed)
5. Gemini returns JSON itinerary: `{ title, days: [{day, title, morning, afternoon, evening, stay, tips}] }`
6. Express streams the response back using Transfer-Encoding: chunked
7. Next.js client renders ItineraryCard components as chunks arrive (live typing)
8. User clicks 'Save This Trip' → POST `/api/v1/ai/save-itinerary`
9. Itinerary stored in `saved_itineraries` table
10. Appears in `/dashboard/user/trips`

9.2 Gemini Prompt Engineering Template

STRUCTURED GEMINI PROMPT (Express controller)

```
const prompt = `You are Saarthi, an expert Bihar travel guide.  
Create a ${days}-day itinerary for Bihar, India.  
Traveller profile: ${group_type}, budget: ${budget} INR/day per person,  
interests: ${interests.join(', ')}, travelling from: ${from_location}.
```

Respond ONLY with valid JSON in this exact schema, no markdown, no preamble:

```
{ title: string, days: Array<{ day: number, title: string,  
  morning: string, afternoon: string, evening: string,  
  stay: string, estimated_cost: number, tips: string }> }
```

```
Include real Bihar destinations: Bodh Gaya, Nalanda, Rajgir, Vaishali,  
Patna Sahib, Vikramshila, Madhubani, etc. Be specific and practical.`
```


10. Detailed Sprint Plan — Week by Week

The sprint is designed for AI-accelerated development using Claude or Cursor. Each task includes the exact files to create or modify, enabling direct copy-paste into an AI prompt. Total estimated hours assume an intermediate developer using AI code generation (roughly 2x faster than writing from scratch).

Week 1: Foundation — Setup, Auth & Database (Est. 30–36 hrs)

#	Task	Description	Files	Hrs	Status
1	Mono-repo scaffold	Create /api (Express+TS) and /frontend (Next.js 14) in single GitHub repo. Configure ESLint, Prettier, shared /types package. Set up .env files.	package.json, tsconfig.json	3	PLANNED
2	Supabase schema migration	Write SQL migration for all 10 tables. Enable RLS on all tables. Create indexes on slug, user_id, service_id. Set up triggers for updated_at and review_count.	supabase/migrations/001_init.sql	4	PLANNED
3	Express skeleton	Set up Express server.ts with helmet, cors, morgan, rate-limiter. Create route files skeleton. Configure Supabase client with service_role key.	src/server.ts, src/config/	2	PLANNED
4	JWT auth middleware	Write authenticate.ts middleware (verify access token from cookie). Write authorize.ts factory (authorize('admin','superadmin')). Add unit tests.	src/middleware/authenticate.ts	3	PLANNED
5	Auth controller (register + verify)	POST /register: hash password, insert user, send OTP via Supabase. POST /verify-otp: verify, sign JWT pair, set httpOnly cookies.	src/controllers/auth.controller.ts	4	PLANNED
6	Auth controller (login + refresh + logout)	POST /login, /refresh (rotate tokens), /logout (delete refresh token). POST /forgot-password and /reset-password.	src/controllers/auth.controller.ts	3	PLANNED
7	Google OAuth flow	GET /auth/google redirect, GET /auth/google/callback — use Supabase Auth OAuth, extract user, sign JWT, set cookies.	src/routes/auth.routes.ts	3	PLANNED
8	Next.js auth pages	Build /login, /register, /verify-otp pages with React Hook Form + Zod. useAuth hook. Redirect if already logged in. Use shadcn/ui Form component.	app/(auth)/	4	PLANNED
9	JWT cookie reading (Next.js)	Server-side auth utility: reads JWT from request cookie in Server Components. Redirects to /login if	lib/auth.ts	2	PLANNED

#	Task	Description	Files	Hrs	Status
		invalid. Used in all protected layout.tsx files.			
10	API client setup	Axios instance with baseURL=EXPRESS_URL, withCredentials:true. Axios response interceptor: on 401 → call /refresh → retry original request.	lib/api-client.ts	2	PLANNED

Week 2: Content, CMS & Provider Panels (Est. 36–42 hrs)

#	Task	Description	Files	Hrs	Status
11	Destinations CRUD API	GET /, GET /:slug, POST /, PUT /:id, DELETE /:id. Supabase queries. Zod validation schema. Admin auth guard on write routes.	routes/destinations, services/	3	PLANNED
12	Packages CRUD API	Same pattern as destinations. Include itinerary JSONB handling. GET with destination_ids join to names.	routes/packages, services/	3	PLANNED
13	Homestays CRUD API	Full CRUD + GET /my (provider listings) + PATCH /:id/availability. host_id = req.user.id on create.	routes/homestays, services/	3	PLANNED
14	Transports & Guides API	CRUD for both. PATCH /guides/:id/verify (admin only — sets is_verified=true). GET /guides/pending (admin).	routes/transports, routes/guides	3	PLANNED
15	Public pages — Destinations	Next.js /destinations (listing, search, filter by category). /destinations/[slug] (article with map, highlights). generateMetadata for SEO.	app/(public)/destinations/	4	PLANNED
16	Public pages — Packages + Homestays	Build /packages (grid with modals) and /homestays (list + filter by location + amenity). /homestays/[slug] with gallery.	app/(public)/packages/, homestays/	4	PLANNED
17	Admin CMS panel	/dashboard/admin/cms/* — DataTable for each resource (TanStack Table). Add/Edit/Delete with modal form. shadcn/ui Sheet for edit drawer.	app/(dashboard)/admin/	5	PLANNED
18	Provider dashboards	/dashboard/provider/[type] — shows own listings, toggle availability, view incoming bookings. Shared layout with role check.	app/(dashboard)/provider/	4	PLANNED
19	Upload endpoint + Supabase Storage	POST /api/v1/upload/signed-url — generates Supabase Storage signed upload URL. Client uploads direct to Supabase. Three buckets: destinations, homestays, profiles.	routes/upload.routes.ts	3	PLANNED
20	Next.js image integration	Use next/image with Supabase Storage domain in next.config.js.	next.config.js, all image components	2	PLANNED

#	Task	Description	Files	Hrs	Status
		BlurDataURL placeholder for all listing cards.			

Week 3: Bookings, Payments & Reviews (Est. 34–40 hrs)

#	Task	Description	Files	Hrs	Status
21	Razorpay order creation	POST /payments/create-order: validate service availability, call razorpay.orders.create, insert pending booking, return order_id.	src/controllers/payment.controller.ts	4	PLANNED
22	Razorpay webhook handler	POST /payments/webhook: verify HMAC-SHA256 signature, update booking status=confirmed, trigger email via Resend.	src/controllers/payment.controller.ts	4	PLANNED
23	Booking Next.js flow	BookingForm component: dates, guests, total price calc. Razorpay checkout.js modal. Polling loop until booking confirmed.	components/booking/, app/confirm/	4	PLANNED
24	User bookings dashboard	/dashboard/user/bookings — list with service_type icons, status badges, cancel button (pending only). Booking detail modal.	app/(dashboard)/user/bookings/	3	PLANNED
25	Admin bookings panel	Admin view of all_bookings. Filter by status, service_type, date. PATCH status. One-click refund via POST /payments/refund/:id.	app/(dashboard)/admin/bookings/	3	PLANNED
26	Review system API	POST /reviews (verify booking_id exists and is completed). GET /:type/:id. DELETE own. Aggregate rating update trigger in Supabase.	routes/reviews.routes.ts, trigger SQL	4	PLANNED
27	Review UI components	StarRating click input. ReviewForm with React Hook Form. ReviewList with verified badge. Add to /homestays/[slug] and /packages/[slug].	components/review/	3	PLANNED
28	Email notifications	Set up Resend. Booking confirmation email, booking cancellation email. HTML email templates using react-email.	src/services/email.service.ts	3	PLANNED

Week 4: AI, Maps, SEO & Production Deploy (Est. 28–34 hrs)

#	Task	Description	Files	Hrs	Status
29	Saarthi — server-side Gemini	POST /ai/generate-itinerary: build structured prompt, call Gemini, stream response. POST /ai/save-itinerary. GET /ai/saved.	routes/ai.routes.ts, controllers/ai.ts	4	PLANNED
30	Saarthi — Next.js UI	SaarthiForm (preferences), streaming ItineraryCard render, Save button, SavedTrips list in dashboard. Match original Saarthi UI design.	app/(dashboard)/ai-planner/, components/ai/	4	PLANNED
31	Google Maps integration	Dynamic import Maps JS API in Client Components. DestinationMap and HomestayMap components. Overview map on /destinations with all pins.	components/destination/DestinationMap.tsx	3	PLANNED
32	Supabase RLS hardening	Write and test all RLS policies from Section 5.11. Use supabase test (pg_tap) to verify policies. Disable service_role bypass on frontend.	supabase/policies/	3	PLANNED
33	SEO — generateMetadata	Dynamic metadata for all detail pages using generateMetadata(). JSON-LD TouristAttraction + LodgingBusiness schemas. OG image via @vercel/og.	all [slug]/page.tsx files	3	PLANNED
34	Performance optimization	next/image, lazy loading, Suspense boundaries on all data-fetching sections, TanStack Query prefetch on server, ISR revalidate=3600 on listing pages.	All page and layout files	2	PLANNED
35	Vercel + Railway deployment	Vercel project for frontend (env vars, custom domain). Railway project for Express (Dockerfile, PORT env). Supabase connection string.	vercel.json, Dockerfile, .env.production	3	PLANNED

#	Task	Description	Files	Hrs	Status
36	GitHub Actions CI/CD	Lint + typecheck on PR. Deploy frontend to Vercel on push to main. Deploy API to Railway. Supabase migration run on deploy.	/.github/workflows/	2	PLANNED

11. Environment Variables Reference

11.1 Express API (.env)

Variable	Description
PORT	Express server port (default 8000)
NODE_ENV	development production
SUPABASE_URL	Your Supabase project URL (https://xxx.supabase.co)
SUPABASE_SERVICE_ROLE_KEY	Service role key — full DB access, NEVER expose on client
JWT_ACCESS_SECRET	Random 64-char secret for signing access tokens
JWT_REFRESH_SECRET	Different random 64-char secret for refresh tokens
JWT_ACCESS_EXPIRY	15m
JWT_REFRESH_EXPIRY	7d
RAZORPAY_KEY_ID	Razorpay API key ID
RAZORPAY_KEY_SECRET	Razorpay API key secret
RAZORPAY_WEBHOOK_SECRET	Webhook signing secret from Razorpay dashboard
GEMINI_API_KEY	Google AI Studio API key for Gemini
RESEND_API_KEY	Resend email API key
CLIENT_URL	Frontend URL for CORS (https://biharyaatra.vercel.app)
GOOGLE_CLIENT_ID	Google OAuth 2.0 client ID
GOOGLE_CLIENT_SECRET	Google OAuth 2.0 client secret

11.2 Next.js Frontend (.env.local)

Variable	Description
NEXT_PUBLIC_API_URL	Express API base URL (https://api.biharyaatra.com/api/v1)
NEXT_PUBLIC_RAZORPAY_KEY_ID	Razorpay key_id — safe to expose (public key only)
NEXT_PUBLIC_GOOGLE_MAPS_KEY	Maps JS API key — restrict to your domain in GCP console

NEXT_PUBLIC_SUPABASE_URL

Supabase project URL (for Storage public URLs only)

NEXT_PUBLIC_SUPABASE_ANON_KEY

Supabase anon key — only for Storage public reads

12. Deployment Architecture

Service	Platform
Next.js Frontend	Vercel — zero-config Next.js deployment, global edge CDN, preview deployments on every PR
Express.js API	Railway — auto-deploys from GitHub, managed Node.js, scales to zero on free tier, custom domain
PostgreSQL Database	Supabase Cloud — managed Postgres, daily backups, point-in-time recovery, global CDN for Storage
Media / Images	Supabase Storage — S3-compatible, direct upload via signed URLs, public CDN for image delivery
CI/CD	GitHub Actions — lint + typecheck on PR, deploy on merge to main for both Vercel and Railway
DNS / Domain	Cloudflare — DNS management, DDoS protection, free SSL, redirect www to apex domain
Email	Resend — transactional email API, 3000 free emails/month, custom domain sending
Monitoring	Vercel Analytics (frontend) + Railway Metrics (API) + Supabase Dashboard (DB)

12.1 Deployment Checklist

1. Create Supabase project → run migration SQL → copy SUPABASE_URL and service_role key
2. Create Railway project → connect GitHub repo → set all Express .env vars → note API URL
3. Create Vercel project → connect GitHub repo → set all Next.js .env vars → set custom domain
4. Configure CORS in Express: CLIENT_URL = Vercel deployment URL
5. Add Vercel domain to Google Cloud Console (Maps API + OAuth) as allowed origin
6. Add Express webhook URL to Razorpay dashboard (Settings → Webhooks)
7. Run Supabase RLS policy SQL → verify with test queries
8. Configure Cloudflare DNS: A record for api.biharyaatra.com → Railway IP
9. Test full booking flow end-to-end with Razorpay test credentials
10. Run Lighthouse on /destinations, /homestays, / → target score ≥ 88 mobile

13. Time Estimate & Effort Breakdown

The following table breaks down effort by module with realistic estimates for AI-assisted development. AI tools (Claude, Copilot, Cursor) provide approximately 2–3x productivity gain on code generation tasks. The estimates below already account for this multiplier.

Module	Solo Dev (AI)	2 Dev (AI)	Without AI	Complexity
Project setup + mono-repo + CI/CD	4–5 hrs	2–3 hrs	8–10 hrs	Low
Database schema + migrations	3–4 hrs	2–3 hrs	6–8 hrs	Medium
JWT Auth (register, login, refresh, OAuth)	8–10 hrs	4–5 hrs	16–20 hrs	Medium
Destinations + Packages API + Pages	8–10 hrs	4–5 hrs	16–20 hrs	Low
Homestays + Transport + Guides API + Pages	8–10 hrs	4–6 hrs	18–22 hrs	Medium
Admin CMS panel	8–10 hrs	5–6 hrs	18–24 hrs	High
Provider dashboards	6–8 hrs	3–4 hrs	12–16 hrs	Medium
Razorpay payment + webhook flow	6–8 hrs	3–4 hrs	14–18 hrs	High
Bookings management (user + admin)	5–6 hrs	3–4 hrs	10–14 hrs	Medium
Review & rating system	5–6 hrs	3–4 hrs	10–14 hrs	Medium
Saarthi AI planner (streaming)	5–6 hrs	3–4 hrs	10–14 hrs	Medium
Google Maps integration	3–4 hrs	2–3 hrs	6–10 hrs	Low
Supabase Storage + image upload	3–4 hrs	2–3 hrs	6–8 hrs	Low
SEO + generateMetadata + JSON-LD	3–4 hrs	2–3 hrs	6–8 hrs	Low
Supabase RLS policies + security	4–5 hrs	2–3 hrs	8–12 hrs	High
Email notifications (Resend)	2–3 hrs	1–2 hrs	4–6 hrs	Low
Performance optimization + testing	4–5 hrs	2–3 hrs	8–12 hrs	Medium
Deployment + CI/CD + DNS	4–5 hrs	2–3 hrs	8–10 hrs	Medium
TOTAL ESTIMATE	96–118 hrs	54–74 hrs	192–246 hrs	21–28 Days

KEY INSIGHT: AI Advantage

Without AI tools: 192–246 hours ≈ 12–15 weeks for 1 developer (full-time)
With AI tools (1 dev): 96–118 hours ≈ 3–4 weeks (full-time, 8 hrs/day)
With AI tools (2 devs): 54–74 hours ≈ 2–2.5 weeks (parallel sprints)

AI tools save approximately 100+ hours on this project alone.
The largest savings are in boilerplate (auth, CRUD), types, and SQL schemas.

14. Risk Register

Risk	Impact	Prob.	Mitigation
Supabase RLS misconfiguration exposes data	HIGH	MED	Write pg_tap tests for every policy. Never use service_role key in frontend. Test as each role before deploy.
Razorpay webhook missed or duplicate	HIGH	LOW	Idempotency key on booking insert. Webhook retry handled via Razorpay dashboard. Log all webhook events.
JWT refresh token not rotated correctly	HIGH	LOW	Token family rotation: invalidate entire family on reuse detection. Store refresh tokens in DB for revocation.
Railway Express server cold start latency	MED	MED	Keep-alive ping every 5 min on free tier. Upgrade to hobby plan (\$5/mo) for persistent containers.
Gemini AI rate limit during peak usage	MED	LOW	Request debounce on Saarthi form. Cache last itinerary. Exponential backoff with user-visible retry prompt.
Supabase Storage public URL leaks private images	MED	LOW	Use signed URLs for private content. Only profile photos are public. Homestay images: signed URLs on booking confirmation.
Next.js App Router caching serves stale data	MED	MED	Use revalidatePath() / revalidateTag() after mutations. Set appropriate revalidate=3600 on listing pages.
AI-generated code type mismatches	LOW	MED	Always define TypeScript interfaces first, then ask AI to implement. Run tsc --noEmit on every AI output before commit.

15. Conclusion & Immediate Next Actions

Bihar Yaatra v3.0 is designed to be a production-grade, scalable, and maintainable tourism platform. The Next.js + Express + Supabase stack provides a clear separation of concerns, full TypeScript safety end-to-end, and a deployment architecture that starts free and scales gracefully with real traffic.

Using AI-assisted development (Claude, Cursor, or GitHub Copilot) with the patterns and prompt templates defined in this document, a single focused developer can deliver the complete platform in 21–28 days. Two developers working in parallel on frontend and backend sprints can cut this to 14–18 days.

Start Today — Day 1 Checklist

11. Create GitHub mono-repo with /api and /frontend directories
12. Run `npx create-next-app@latest frontend --typescript --tailwind --app`
13. Run `npx express-generator-typescript api` — set up `server.ts` from scratch
14. Create Supabase project at `supabase.com` → copy URL and keys to `.env`
15. Run the Week 1 SQL migration (Sprint Task #2) in Supabase SQL editor
16. Install `shadcn/ui`: `npx shadcn@latest init` in /frontend
17. Build the JWT auth endpoints (Sprint Tasks #4–6) — this unlocks everything else
18. Test with Postman: register → verify OTP → login → GET /me → logout

Bihar Yaatra v3.0 — Production Launch Checklist

- ☐ Supabase schema migrated + RLS policies active
- ☐ JWT auth complete (email + OTP + Google OAuth)
- ☐ All 5 content types with CRUD API + Next.js pages
- ☐ Admin CMS panel live
- ☐ Provider dashboards (homestay, guide, transport)
- ☐ Razorpay payment + webhook flow tested end-to-end
- ☐ Review & rating system on all service types
- ☐ Saarthi AI planner streaming + save to profile
- ☐ Google Maps on all detail pages
- ☐ SEO generateMetadata + JSON-LD on all pages
- ☐ Lighthouse score ≥ 88 on mobile
- ☐ Vercel (frontend) + Railway (API) deployed
- ☐ GitHub Actions CI/CD pipeline active
- ☐ Custom domain live — `biharyaatra.com`
- ☒ **PLATFORM LIVE ✓ — 21-28 Days from Day 1**